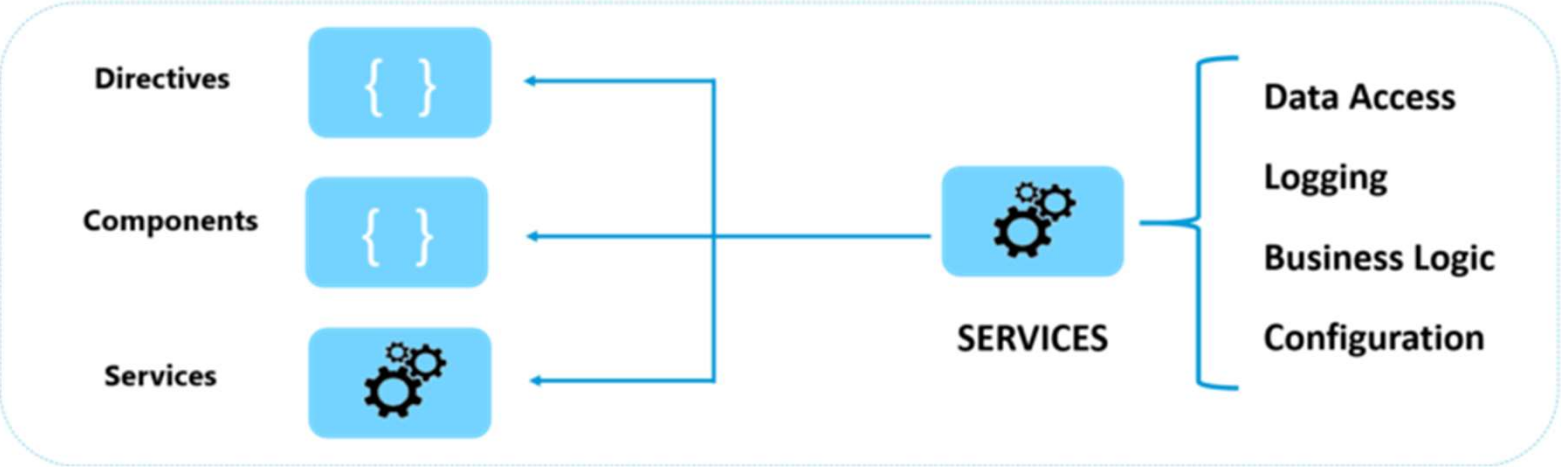
Services

Services

- *Service* is a broad category encompassing any value, function, or feature that your application needs
- A service is typically a class with a well-defined purpose. Anything can be a service
- Examples include:
  - logging service
  - data service
  - tax calculator



Services

- A service is typically a class with a narrow, well-defined purpose
- It should do something specific and do it well
- Ideally, a component's job is to present properties and methods for data binding, in order to mediate between the view and the application logic
- A component does not need to define things like how to fetch data from the server, validate user input, or log directly to the console
- Instead, it can delegate such tasks to services
- By defining that kind of processing task in an injectable service class, you make it available to any component

Creating Service

- Create service with the command

```
ng  g s hello
```

- Register the service with the module as a provider

```
@NgModule({
  providers: [
  HelloService
 ],
 ……
```

- Use dependency injection in the constructor of a component

```
constructor(private service:HelloService) {}
```

Scope of the Service

- When you register the service at module level, the same instance of the service is injected in all the components of the module
- To make separate instance available for the component, register it with component

```
@Component({
  selector:    'app-courses',
  templateUrl: './course.component.html',
  providers:  [ HelloService ]
})
```

Dependency Injection

- DI is wired into the Angular framework and used everywhere to provide new components with the services or other things they need

- Components consume services
- So, we can *inject* a service into a component, giving the component access to that service class

- To define a class as a service in Angular, use the @Injectable() decorator to provide the metadata that allows Angular to inject it into a component as a *dependency*

Injector hierarchies

INJECTOR HIERARCHIES	DETAILS
ModuleInjector hierarchy	Configure a ModuleInjector in this hierarchy using an @NgModule() decorator.
ElementInjector hierarchy	Created implicitly at each DOM element. An ElementInjector is empty by default unless you configure it in the providers property on @Directive() or @Component().

Searching order for injectors

- Component
- Parent component
- Parent module `providedIn: “root",`
- Platform `providedIn: “platform",`
- Null Injector (top level)

DI Decorators

- @self -- use only element injector

  constructor(@Self() public service: LogService) { }

- @Optional -- not injected if component does not have injector

  constructor(@Optional() public service: LogService) { }

- @Host --
  - designates a component as the last stop in the injector tree when searching for providers. Even if there is a service instance further up the tree, Angular won't continue looking

  constructor(@Host() @Optional() public service: LogService) { }

@SkipSelf -- look for injector from above the component

Providers

- DI Angular Providers allows us to register classes, functions, or values (dependencies) with the Angular Dependency Injection system.
- The Providers are registered using the token.
- The tokens are used to locate the provider.
- We can create three types of the token. Type Token, string token & Injection Token.
- The Provider also tells the Angular Injector how to create the instance of dependency.
- There are four ways by which you can create the dependency:
  - Class Provider (useClass),
  - Value Provider (useValue),
  - Factory Provider (useFactory),
  - Aliased Class Provider (useExisting).

Providers

- We register the services participating in the dependency injections in the Providers metadata. There are two ways by which we can do it.

- Register directly in the Providers array of the @NgModule or @Component or in @Directive.
- Or use the providedIn property of the @Injectable decorator.
- The Angular creates an Injector for each component/directive it creates

Configuring Providers

Type token

```
providers: [ProductService]
```

or

```
providers: [{provide:ProductService,
                                useClass:ProductService}]
```

- This will create object and inject it

Configuring Providers

String token

- Instead of using a type, we can use a string literal to register the dependency. This is useful in scenarios where the dependency is a value or object etc

```
{provide:'PRODUCT_SERVICE', useClass: ProductService },
 {provide:'USE_FAKE', useValue: true },
 {provide:'APIURL', useValue: 'http://SomeEndPoint.com/api' },


class ProductComponent {
   constructor(@Inject('PRODUCTSERVICE') private prdService:ProductService,
                  @Inject('APIURL') private apiURL:string ) {
}
```

Configuring Providers

Injection Token

- String token creates problem if two developers generate same token
- Angular provides InjectionToken class so as to ensure unique tokens are created

```
export const API_URL= new InjectionToken<string>(‘’);


providers: [
    { provide: API_URL, useValue: 'http://SomeEndPoint.com/api' }
]


constructor(@Inject(API_URL) private apiURL: string) {
}
```

Server Communication

RxJs library

- Reactive programming is an asynchronous programming paradigm concerned with data streams
- RxJS (Reactive Extensions for JavaScript) is a library for reactive programming using observables that makes it easier to compose asynchronous or callback-based code
- Callback is a mechanism to notify after an asynchronous operation is completed
- RxJS offers a number of functions that can be used to create new observables

Observable

- Observables support passing the messages between publishers(Creator of Observables) and subscribers(User of Observables) in your application.
- Creating Observable:

```
var observable = Observable.create((observer:any) => {
    observer.next('Hello World!')
```

- Subscribing to observable

```
observable.subscribe(function logMessage(message:any) {
  console.log(message);
})
```

Observable

```
var observable = new Observable((observer:any) => {
    observer.next('I am number 1')
    observer.next('I am number 2')
    observer.error('I am number 3')
    observer.complete('I am number 4')
 })


 observable.subscribe((message:any)=> {
  console.log(message);
 })  })
```

Observable

- Subscriber has to provide callback methods to handle the three types of notifications it can send:

next	Required. A handler for each delivered value. Called zero or more times after execution starts.
error	Optional. A handler for an error notification. An error halts execution of the observable instance.
complete	Optional. A handler for the execution-complete notification.

How to use Observable

- Subscribe to observable by providing callback functions

```
// Subscribe to begin listening for async result
//format1
data.subscribe({
  next(response) { console.log(response); },
  error(err) { console.error('Error: ' + err); },
  complete() { console.log('Completed'); }
});

//format2
data.subscribe(
  x => console.log('Observer got a next value: ' + x),
  err => console.error('Observer got an error: ' + err),
  () => console.log('Observer got a complete notification')
);
```

RxJS pipe()

- pipe() takes a bunch of RxJS operators as arguments such as filter and map separated by comma and run them in the sequence they are added and finally returns an RxJS Observable
- To get the result we need to subscribe() to the returned Observable.

RxJS Operators

- Operators are functions that build on the observables foundation to enable sophisticated manipulation of collections

<u>map() operator</u>

converts observable into observable of new values

```
import { of } from 'rxjs';
import { map } from 'rxjs/operators';

const nums = of(1, 2, 3);  // creates observable

const squareValues = map((val: number) => val * val);
const squaredNums = squareValues(nums);

squaredNums.subscribe(x => console.log(x));
```

RxJS Operators

max() operator

- The RxJS max() operator takes in an observable and returns an observable with the maximum value.

```
import { of } from 'rxjs';
import { max } from 'rxjs/operators';
let all_nums = of(1, 3, 5, 6, 12, 13, 24, 28);
let final_val = all_nums.pipe(max());
final_val.subscribe(x => console.log("The Max value is "+x));
```

RxJS Operators

filter() operator

- The RxJS filter() operator takes in an observable and a filter function and returns an observable with the filtered values.

```
import { of } from 'rxjs';
import { map, reduce, filter } from 'rxjs/operators';

let test1 = of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
let case1 = test1.pipe(
  filter(x => x % 2 === 0)
)
```

HttpClientModule

- HttpClientModule is required for server interaction
- We need to import the http module to make use of the http service
- Let us consider an example to understand how to make use of the http service

```
......
import { HttpClientModule } from '@angular/common/http';
......
@NgModule({
  declarations: [
    AppComponent
  ],
  imports: [
    BrowserModule,
    HttpClientModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
```

HttpClient

- HttpClient helps fetch external data, post to it, etc.
- We need to import the http module to make use of the http service
- This service is available as an injectable class

```
import {HttpClient} from '@angular/common/http';
export class DataService {

  constructor(private http: HttpClient) {

  }
```

HttpClient

- HttpClient provides methods for corresponding HTTP methods like
  - Get
  - Post
  - Put
  - Delete etc

- First argument is URL
- Other arguments like headers etc can be added
- These methods return Observable<any>
- Clients can subscribe to the observable to handle response

HttpClient Example

```
this.http.get(“http://host:4200/persons).subscribe(
   (response) => { console.log(response) },
   (err) => console.error('Observer got an error: ' + err),
    () => console.log('Observer got a complete notification')
);
```

HttpClient Example

Using Observer object with next, error and complete properties:

```
this.http.get("http://host:4200/persons) .subscribe(
 {
    next:   (response) => { console.log(response) },
    error:   (err) => console.error('Observer got an error: ' + err),
    complete:  () => console.log('Observer got a complete notification')
 }
);
```

async pipe

- Async Pipe automatically subscribes to an observable to emit the latest values
- It not only subscribes to an observable, but it also subscribes to a promise and calls the then method
- When the components get destroyed, it automatically unsubscribes them to reduce memory leaks.

- Async Pipe makes the rendering of data from observable and promise easier.
- For promises, it automatically calls the then method.
- For observables, it automatically calls subscribe and unsubscribe.:

async pipe

- Example using subscribe:

```
products:Product[ ];

 public getProductsUsingSubscribeMethod() {

  this.productService.getAllProducts().subscribe(
          (data: any) => {
          this.products =data;
           })
 }
```

In Template:

```
          <tr *ngIf='let p of products'>..... </tr>
```

async pipe

- Example using async:

```
Observable<Product[]> productsObs;
 public getProductsUsingAsyncPipe() {
    this.productsObs =this._productService.getAllProducts();
 }
```

In Template:

```
<tr *ngIf='let p of productsObs|async '>..... </tr>
```

Http interceptors

- Interceptors are a unique type of Angular Service that we can implement.
- Interceptors allow us to intercept incoming or outgoing HTTP requests using the HttpClient
- By intercepting the HTTP request, we can modify or change the value of the request.
- The class has to implement

```
interface HttpInterceptor {
  intercept(req: HttpRequest<any>, next: HttpHandler): Observable<Http
Event<any>>
}
```

- The class has to be included in AppModule providers

Http interceptors example

```
export class TokenInterceptor implements HttpInterceptor {
constructor(public auth: AuthService) {}
intercept(request: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {

   request = request.clone({
     setHeaders: {
       Authorization: `Bearer ${this.auth.getToken()}`
     }
   });
              return next.handle(request);
  }
}
```

<u>In AppModule</u>

```
   providers: [    { provide: HTTP_INTERCEPTORS,   useClass: TokenInterceptor,     multi: true    }
```